Week 9: Graded Mini Project

Project title: House Sales Data Analysis

Objective: Analyse housing data to identify key factors influencing sale prices through exploratory analysis, feature engineering, and predictive modeling.

Submitted by: Arpit Jain | Notebook: Week 9_Graded Mini Project_Jain.ipynb (full code and outputs)

Submission Guidelines

- Please document your response on the following pages.
- Include your code (as screenshots or snippets), visualisations, insights, and model evaluation.
- Save your work as a PDF named: **Week 9_Graded Mini Project_[Your Last Name].pdf.**

Your submission will be considered complete ONLY if it follows the template, addresses all project tasks clearly, and is submitted on time in the correct format.

Note: This is a required activity and counts towards programme completion.

Reflect on the tasks and respond to the following questions.

Task 1: Data Cleaning and Preprocessing

1.1 Handling Missing Values: Identify and handle missing values appropriately.

19 of the 81 columns have missing values. Following the data description, a blank usually means **the facility does not exist** (for example, no garage), so each column was treated according to what the gap means:

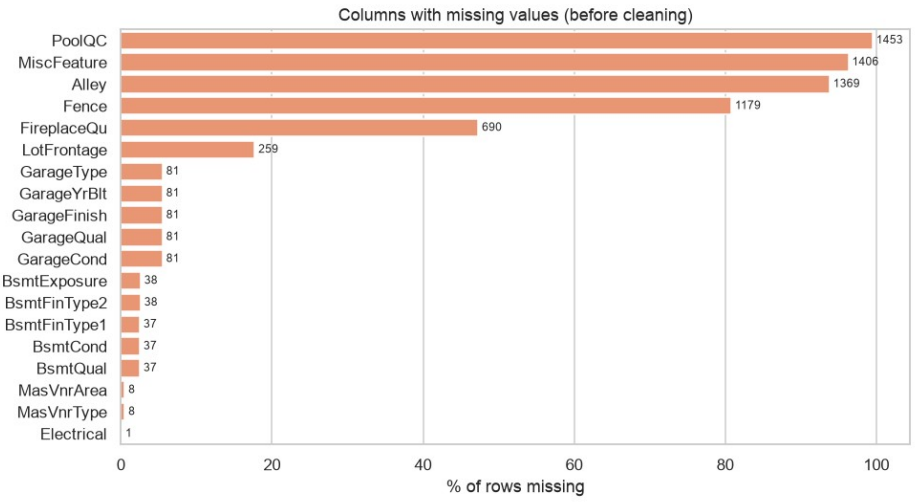


Figure 1 - Columns with missing values (count shown at bar end)

Group	Columns	Treatment
Categorical, facility absent	PoolQC, MiscFeature, Alley, Fence, FireplaceQu, Garage Type/Finish/Qual/Cond, Bsmt Qual/Cond/Exposure/FinType1/FinType2, MasVnrType	Filled with category "None"
Numeric field of that facility	GarageYrBlt, MasVnrArea (and other Bsmt/Garage numerics)	Filled with 0 (as instructed)
Genuine gaps (facility exists)	BsmtExposure (house 949), BsmtFinType2 (house 333) - both have a basement; Electrical (house 1380)	Mode from training data: "No", "Unf", "SBrkr"
Measurement, not a facility	LotFrontage (259 rows, 17.7%)	Median LotFrontage of the same Neighborhood (training data)

For Use only

- **Pandas detail:** by default pandas reads the text "None" as missing. Here "None" is a real MasVnrType category, so the file is read with keep_default_na=False, na_values=['NA',''']. This reduces the MasVnrType gaps from 872 to the 8 that are really missing.
- **LotFrontage is not set to 0:** every lot touches a street, so 0 would be false. Lots in the same neighbourhood have similar frontage, which makes the neighbourhood median a sensible estimate.
- **No leakage:** the medians and modes were learned from the training split only. After cleaning, 0 missing values remain in both train and test.

Code snippet (notebook section 1.1)

```
raw = pd.read_csv("houseprice.csv", keep_default_na=False, na_values=["NA", ""])
df[NONE_COLS] = df[NONE_COLS].fillna("None")          # facility absent
df[ZERO_COLS] = df[ZERO_COLS].fillna(0)               # its numeric fields -> 0
lot_med = train_df.groupby("Neighborhood")["LotFrontage"].median()
d["LotFrontage"] = d["LotFrontage"].fillna(d["Neighborhood"].map(lot_med))
```

1.2 Converting Categorical Variables: Encode categorical variables using suitable methods (e.g., one-hot encoding).

Encoding was chosen per column type, so that order is preserved where it exists and never invented where it doesn't:

Type	Columns	Method	Justification
Ordered quality scales	ExterQual, ExterCond, BsmtQual, BsmtCond, HeatingQC, KitchenQual, FireplaceQu, GarageQual, GarageCond, PoolQC	Ordinal: None=0, Po=1, Fa=2, TA=3, Gd=4, Ex=5	Clear order; one column keeps it
Other ordered scales	BsmtExposure, BsmtFinType1/2, GarageFinish, Functional, LotShape, LandSlope, PavedDrive, CentralAir, Street, Utilities	Ordinal by rank	Same reasoning
Binned features	AgeBand, QualBand, SizeBand, EraBuilt, LotBand	Ordinal 0-3	Bins are ordered
Nominal (no order)	23 columns: MSZoning, Neighborhood, MSSubClass, HouseStyle, Exterior1st/2nd, SaleType, Foundation, ...	One-hot encoding	Integers would imply a false order

- **MSSubClass** is stored as a number (20, 60, 120...), but it is a **code** for the dwelling type, so it was converted to text and one-hot encoded.
- **Train/test consistency:** one-hot columns were created from the training categories, and the test set was re-indexed to match. A category that only appears in the test set becomes all-zero instead of breaking the model.
- **Result:** 21 ordinal columns + 5 bins + 23 one-hot columns give a final model matrix of **260 features** (1,166 training rows, 292 test rows).

Code snippet (notebook section 1.2)

```
d["MSSubClass"] = "SC" + d["MSSubClass"].astype(str)      # code, not a quantity
for col, mapping in ORDINAL_MAPS.items():                  # Ex=5 ... Po=1, None=0
    d[col] = d[col].map(mapping)
X_train = pd.get_dummies(tr, columns=NOMINAL, dtype=int)
X_test = pd.get_dummies(te, columns=NOMINAL, dtype=int).reindex(columns=X_train.columns,
fill_value=0)
```

1.3 Normalising Numerical Features: Normalise numerical features to ensure they are on a comparable scale.

- **Step 1, log transform:** 39 continuous columns with |skewness| > 0.75 were transformed with log(1 + x). For example, LotArea skew dropped from 12.3 to 0.0 and LotArea_per_SF from 9.0 to 0.8.
- **Step 2, standardise:** every continuous column was scaled to mean 0 and standard deviation 1 with StandardScaler. The scaler was **fitted on the training set only** and then applied to the test set.
- **Dummy columns:** one-hot 0/1 columns were left unscaled, because scaling them adds nothing.
- **Target:** SalePrice itself was log-transformed (skew 1.88 to 0.12). Models predict log price, which is converted back to dollars for the metrics.

For learner use only

- **Note:** tree boosters are insensitive to scale. Normalisation matters for the Ridge baseline, and using one matrix for all models keeps the comparison consistent.

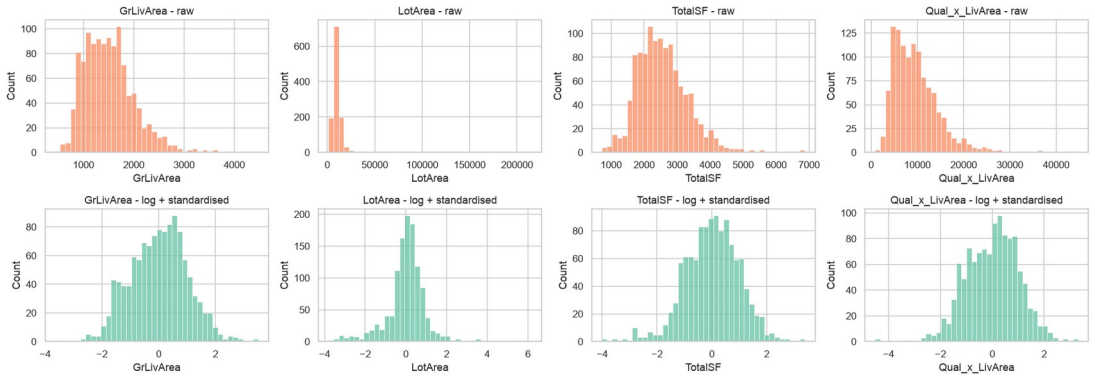


Figure 2 - Raw (top) vs log + standardised (bottom) distributions

Code snippet (notebook section 1.3)

```
skewed = [c for c in cont_cols if abs(X_train[c].skew()) > 0.75 and (X_train[c] >= 0).all()]
X_train_n[skewed] = np.log1p(X_train_n[skewed]); X_test_n[skewed] = np.log1p(X_test_n[skewed])
scaler = StandardScaler().fit(X_train_n[cont_cols])          # fit on TRAIN only
X_train_n[cont_cols] = scaler.transform(X_train_n[cont_cols])
X_test_n[cont_cols] = scaler.transform(X_test_n[cont_cols])
```

1.4 **Outlier Detection:** Identify and handle outliers in the dataset.

Two methods were used. The first is a statistical IQR scan (1.5 x IQR rule) of the key numeric columns. The second is a domain check of price against the strongest driver, living area.

Column	IQR upper fence	Rows flagged	Max value
SalePrice	340,038	61	755,000
GrLivArea	2,748	31	5,642
LotArea	17,674	69	215,245
TotalBsmtSF	2,052	61	6,110
1stFlrSF	2,155	20	4,692
GarageArea	938	21	1,418
LotFrontage	112	88	313
MasVnrArea	415	96	1,600

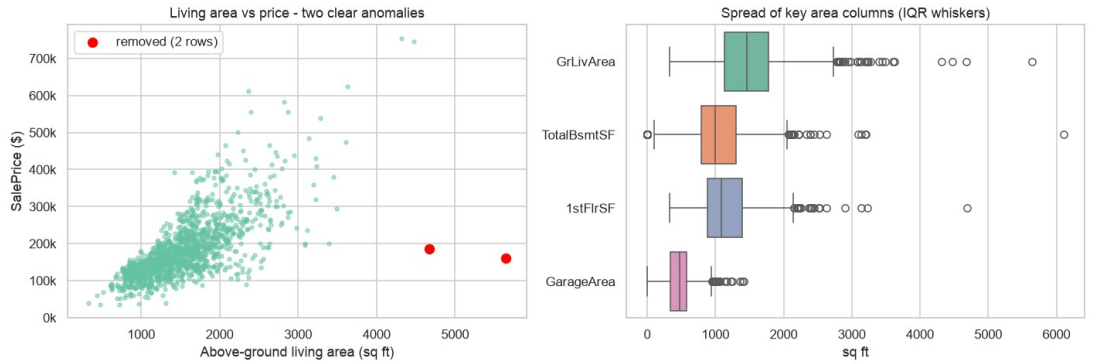


Figure 3 - Living area vs price (left) and spread of area columns (right)

- **IQR-flagged rows were kept.** Most are simply large or expensive houses, which are real data and the top of the market the model must learn. Their influence was reduced with the **log transforms** described in 1.3.
- **Two rows were removed (Ids 524, 1299).** These are houses of more than 4,500 sq ft with quality 10 that sold for only $160k-185k. Both are **Partial** sales in Edwards, meaning they were sold before completion, so the price does not reflect the house. The dataset's author also recommends removing them.
- **The other two very large houses** (over $700k) follow the size-price trend and were kept. This leaves **1,458**

rows.

Task 2: Feature Engineering

2.1 Creating Interaction Features: Create meaningful interaction features that may help improve model performance.

17 features were engineered, combining columns into quantities that buyers actually think about:

Feature	Formula	Why it helps
TotalSF	TotalBsmtSF + 1stFlrSF + 2ndFlrSF	Total usable space
Qual_x_TotalSF / Qual_x_LivArea	OverallQual x TotalSF / x GrLivArea	Size is worth more when quality is high
Kitchen_x_Qual	KitchenQual score x OverallQual	Kitchens drive buyer decisions
Qual_x_Cond	OverallQual x OverallCond	Good materials that are also well kept
Garage_x_Qual	GarageCars x GarageQual score	Capacity only counts if in good shape
TotalBath	Full + 0.5 Half + BsmtFull + 0.5 BsmtHalf	All bathrooms in one count
HouseAge / RemodAge / IsRemodeled	YrSold - YearBuilt / - YearRemodAdd / flag	Age at time of sale; renovation
TotalPorchSF, LotArea_per_SF	Sum of porch areas; LotArea / TotalSF	Outdoor space; land per sq ft
HasGarage, HasBsmt, HasFireplace, Has2ndFlr, HasPool	area/count > 0	Presence of each facility

Evidence that they help. First, the correlations with log price: Qual_x_TotalSF reaches **r = 0.88** and Qual_x_LivArea 0.84, both higher than OverallQual (0.81) or GrLivArea (0.73). Second, an **ablation test**: each tuned booster was cross-validated with and without the interaction features.

Model	CV RMSE (log) without	CV RMSE (log) with	Improvement	Folds improved
XGBoost	0.1146	0.1108	+3.3%	5/5
LightGBM	0.1174	0.1127	+4.0%	5/5
CatBoost	0.1116	0.1089	+2.5%	5/5
Ridge (baseline)	0.1144	0.1146	-0.2%	2/5

The interactions improve **all three boosters** by 2.5-4.0%, and **in every one of the 5 folds**. They are also the top features in all three models (Figure 14). They don't help Ridge, because Ridge works on log-transformed inputs: log(quality) + log(area) already captures the same multiplication.

Code snippet (notebook section 2.1)

```
d["TotalSF"] = d["TotalBsmtSF"] + d["1stFlrSF"] + d["2ndFlrSF"]
d["Qual_x_TotalSF"] = d["OverallQual"] * d["TotalSF"]
d["Kitchen_x_Qual"] = d["KitchenQual"].map(QUAL_MAP) * d["OverallQual"]
d["TotalBath"] = d["FullBath"] + 0.5*d["HalfBath"] + d["BsmtFullBath"] +
0.5*d["BsmtHalfBath"]
d["HouseAge"] = (d["YrSold"] - d["YearBuilt"]).clip(lower=0)
```

2.2 Feature Binning: Bin continuous variables into discrete categories for better interpretability.

Five continuous variables were binned into labelled groups that a non-technical user can read easily. The edges are fixed domain boundaries, except SizeBand, which uses training-set quartiles.

Bin	Edges	Categories (houses, median price)
AgeBand	HouseAge: 0-5 / 6-20 / 21-50 / 51+ yrs	New (308, $226k); Recent (254, $212k); Established (457, $147k); Old (439, $127k)
QualBand	OverallQual: 1-4 / 5-6 / 7-8 / 9-10	Low (141, $105k); Average (771, $143k); Good (487, $222k); Excellent (59, $374k)
SizeBand	GrLivArea quartiles: <1,134 / <1,464 / <1,788 / above	Small (371, $123k); Medium (360, $155k); Large (371, $185k); Very large (356, $240k)
EraBuilt	YearBuilt: <1950 / 1950-79 / 1980-99 / 2000+	Pre-1950 (318, $122k); 1950-1979 (530, $142k); 1980-1999 (224, $194k); 2000+ (386, $224k)
LotBand	LotArea: <7.5k / 7.5-10k / 10-15k / >15k sq ft	Compact (363, $133k); Standard (479, $153k); Large (496, $190k); Estate (120, $219k)

For learner use only

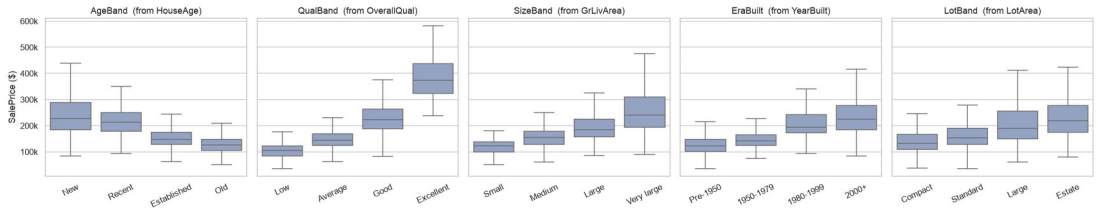


Figure 4 - Price distribution within each bin

- Every bin produces **clearly stepped, monotonic price levels**. QualBand goes from $105k to $374k (a 3.6x range), and AgeBand from $226k for New to $127k for Old. The categories are therefore logical and useful.
- The bins are ordered, so they enter the models as **ordinal codes 0-3**. QualBand appears among the top-15 features of XGBoost and CatBoost, and SizeBand in CatBoost's.

Code snippet (notebook section 2.2)

```
BINS = {"AgeBand":  ("HouseAge",    [-1, 5, 20, 50, np.inf], ["New", "Recent", "Established",
"Old"]),
        "QualBand": ("OverallQual", [0, 4, 6, 8, 10],        ["Low", "Average", "Good",
"Excellent"]), ...}
for new, (src, edges, labels) in BINS.items():
    d[new] = pd.cut(d[src], bins=edges, labels=labels)
```

Task 3: Exploratory Data Analysis (EDA)

3.1 Analysis and Interpretation: Analyse the dataset to answer specific questions:

I. What is the distribution of sale prices?

II. How do different features relate to sale price?

III. Identify trends and patterns in the data.

I. What is the distribution of sale prices?

- **Spread:** prices range from $34,900 to $755,000. The median is $163,000 and the mean $180,933. Half of all houses sell between $130k and $214k.
- **Shape:** the distribution is **right-skewed** (skewness 1.88, kurtosis 6.5). A small number of expensive houses form a long tail and pull the mean above the median.
- **Log transform:** log(1 + price) is close to normal (skewness 0.12), and the Q-Q plot is almost straight (Figure 5). The models therefore predict log price, which makes errors relative: a $20k miss matters more on a $100k house.

II. How do different features relate to sale price?

- **Quality is the strongest driver.** OverallQual has r = 0.82 with log price, and the median price rises at every step from 1 to 10. Kitchen, exterior and basement quality show the same staircase: median $318k for Ex kitchens against $137k for TA.
- **Size comes next.** TotalSF (r = 0.83), GrLivArea (0.73), garage cars and area (0.68 / 0.66) and basement area (0.65) all matter. **Size and quality multiply**: large houses only reach top prices when quality is also high (Figure 6, centre), which is why the interaction features work.
- **Age and renovation:** YearBuilt (r = 0.59) and YearRemodAdd (0.57). Newer houses are worth more.
- **Location:** median prices run from $315k (NridgHt) down to $88k (MeadowV), a 3x gap for location alone (Figure 7).
- **Amenities:**
  - Central air: $168k with it vs $98k without.
  - Fireplace: $190k with one vs $135k with none.

For learner use only

- Garage capacity: $100k with no garage up to $295k for 3 cars.

- **Weak or negative links:** month and year sold, MiscVal and OverallCond on its own barely matter (|r| < 0.05). EnclosedPorch and KitchenAbvGr are slightly negative (r about -0.15), because they are typical of older houses and duplexes.
- **Correlated pairs** carry overlapping information: GarageCars/GarageArea (0.89), GrLivArea/TotRmsAbvGrd (0.83) and TotalBsmtSF/1stFlrSF (0.80). Tree ensembles handle this well.

III. Trends and patterns

- **Prices were stable over time.** The median was $162k-166k from 2006 to 2009, even through the 2008 crisis, with a small dip to $155k in 2010 (a year with only part-year data).
- **Seasonality in volume, not price.** Sales peak in May-July (204-253 a month vs 52-59 in Dec-Feb), but the median price hardly changes by month.
- **Sale condition:** Partial sales of new builds have the highest median ($247k). Abnormal sales (foreclosures) go for $130k, against $160k for Normal sales.
- **New-construction premium:** price per sq ft rises from about $65-85 for pre-1920 homes to $118 for 1950s-70s homes and $139 for the 2000s. Newer homes are worth more per sq ft, not just bigger.
- **Overall pattern:** the highest prices need quality, size and a good neighbourhood together. No single feature is enough.

3.2 **Visualisations:** Create plots and charts to represent your findings clearly.

Key charts are shown below. All plots are in the notebook (sections 3.1-3.3); figures 1-4 appear in the Task 1 and 2 answers above.

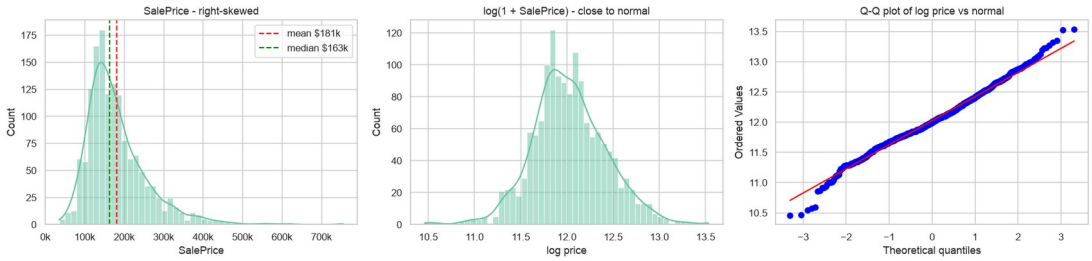


Figure 5 - SalePrice distribution: raw (skewed), log (near-normal), Q-Q plot

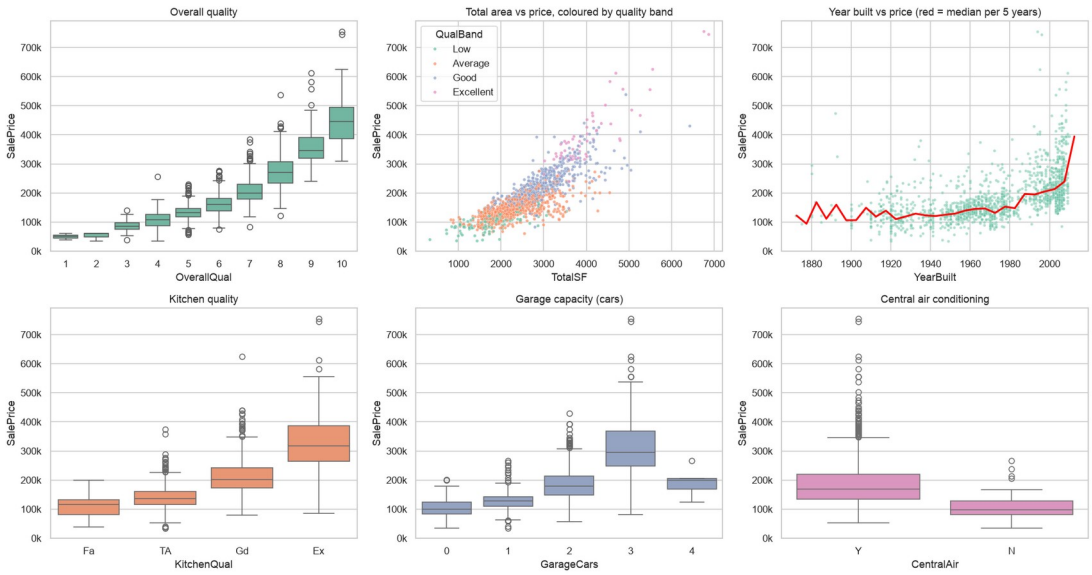


Figure 6 - Price vs quality, total area (by quality band), year built, kitchen, garage, central air

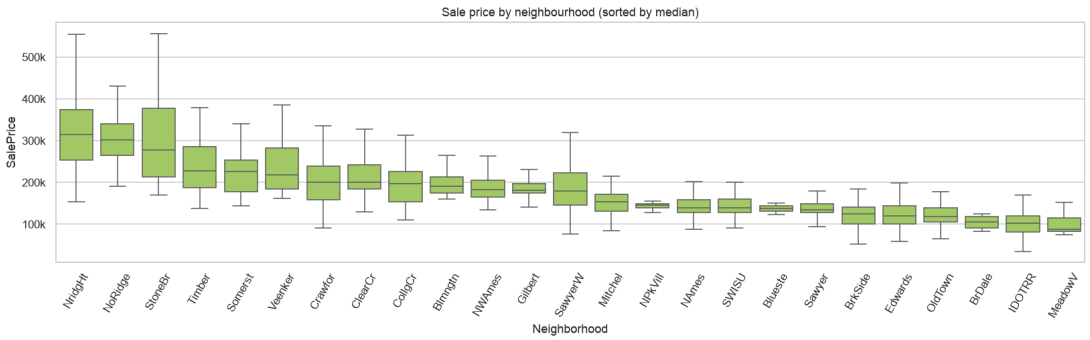


Figure 7 - Price by neighbourhood, sorted by median

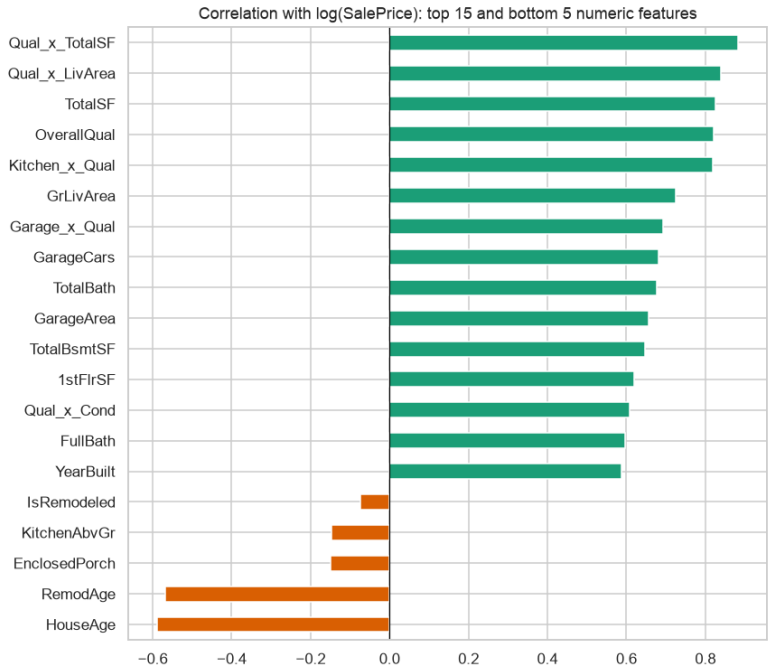


Figure 8 - Correlation of numeric features with log(SalePrice)

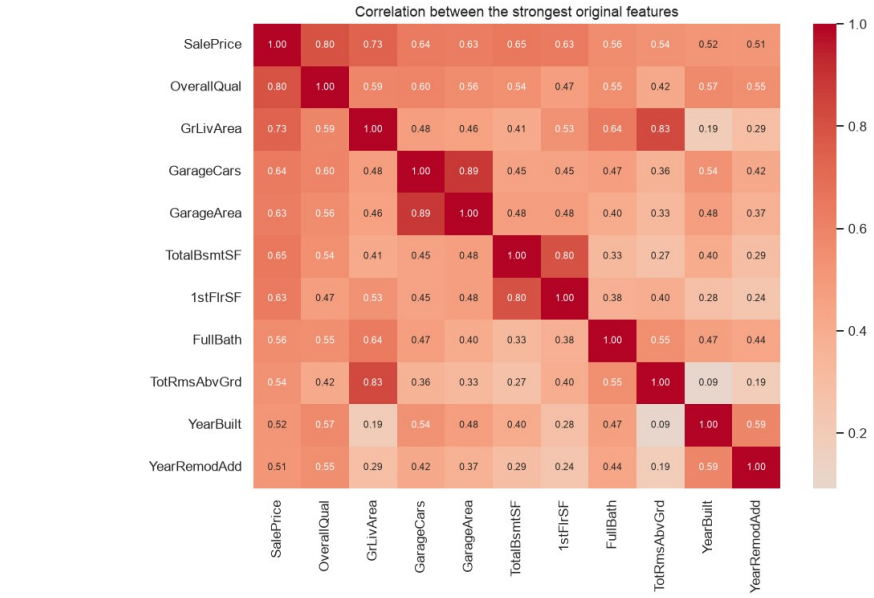


Figure 9 - Correlation between the strongest original features



Figure 10 - Price by year sold, sales volume by month, price by sale condition

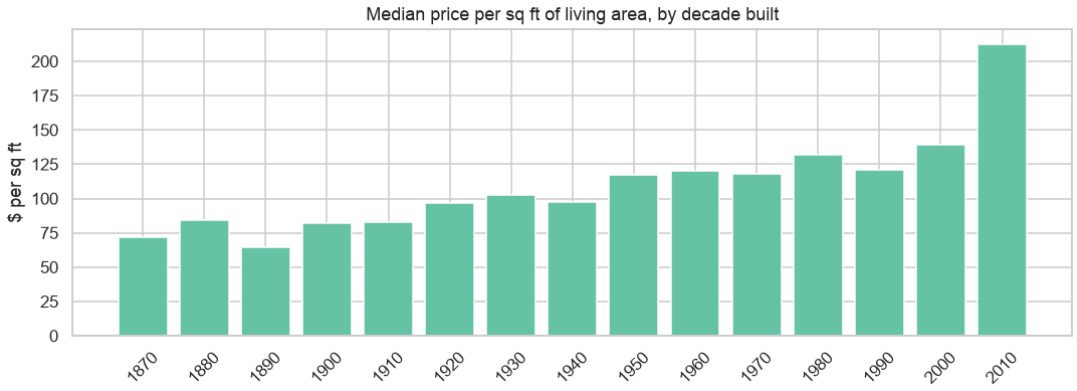


Figure 11 - Price per sq ft by decade built

Task 4: Model Building and Evaluation

I. Build models using XGBoost, LightGBM, and CatBoost to predict sale prices.

II. Evaluate model performance using metrics such as RMSE, MAE, and R^2.

Model Building

- **Data:** 1,458 houses, 260 features, split 80/20 (1,166 train / 292 test, random_state 42). The test set was

only used once, for the final evaluation.

- **Target:** log(1 + SalePrice). Predictions were converted back with expm1 for the dollar metrics.
- **Models:** XGBoost, LightGBM and CatBoost (as required), plus a **Ridge regression baseline** to show what boosting adds. A simple average of the three boosters was also tested.
- **Tuning:** RandomizedSearchCV with 25 parameter combinations x 5-fold CV for each booster, minimising RMSE. The search covered the number of trees, learning rate, tree depth/leaves, row and column subsampling, and L1/L2 regularisation.

Model	Best parameters (from 5-fold CV search)
XGBoost	n_estimators=1000, learning_rate=0.03, max_depth=3, min_child_weight=1, subsample=0.8, colsample_bytree=0.3, reg_alpha=0.001, reg_lambda=1
LightGBM	n_estimators=600, learning_rate=0.03, num_leaves=8, min_child_samples=5, subsample=0.7, colsample_bytree=0.5, reg_lambda=1
CatBoost	iterations=1500, learning_rate=0.05, depth=4, l2_leaf_reg=5, rsm=0.3, subsample=0.7 (Bernoulli)
Ridge	alpha=11.7 (RidgeCV)

All three boosters settled on **shallow trees** (depth 3-4 or 8 leaves), **slow learning rates** and **strong column subsampling** (30-50% of features per tree). With only about 1,200 training houses, many small, randomised trees generalise better than a few deep ones.

Code snippet (notebook section 4)

```
search = {"XGBoost": (XGBRegressor(random_state=SEED, tree_method="hist"), {"n_estimators":
[600, 1000, 1500],
              "learning_rate": [0.02, 0.03, 0.05], "max_depth": [3, 4, 5],
"colsample_bytree": [0.3, 0.5, 0.7], ...}),
          "LightGBM": (LGBMRegressor(random_state=SEED, verbose=-1), {...}),
          "CatBoost": (CatBoostRegressor(random_seed=SEED, verbose=0), {...})}
for name, (est, grid) in search.items():
    rs = RandomizedSearchCV(est, grid, n_iter=25, cv=KFold(5, shuffle=True,
random_state=SEED),
                            scoring="neg_root_mean_squared_error",
random_state=SEED).fit(X_train_n, y_train)
    best_models[name] = rs.best_estimator_
```

Evaluation

Model	CV RMSE (log) ± std	Test RMSE ($)	Test MAE ($)	Test R^2	Test RMSE (log)	MAPE
CatBoost	**0.1089 ± 0.0102**	**18,205**	**13,087**	**0.940**	**0.1100**	**7.9%**
Average of 3 boosters	-	18,528	13,319	0.938	0.1126	8.1%
XGBoost	0.1108 ± 0.0115	19,427	13,969	0.932	0.1147	8.3%
LightGBM	0.1127 ± 0.0105	19,297	13,634	0.933	0.1177	8.3%
Ridge (baseline)	0.1146 ± 0.0088	19,088	13,775	0.934	0.1139	8.2%

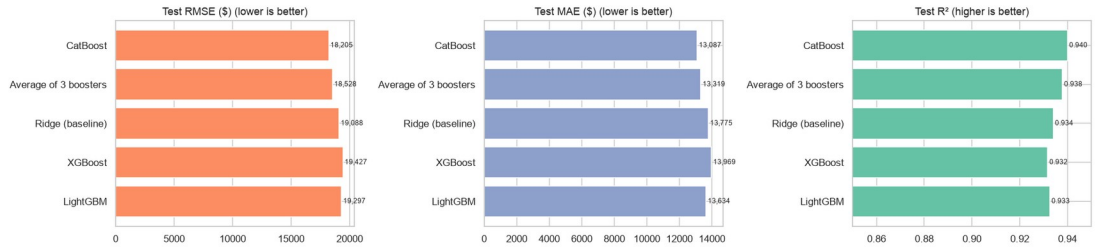


Figure 12 - Test RMSE, MAE and R^2 by model

- **CatBoost is the best model**, both in CV (RMSE 0.109 on log price) and on the unseen test set: **RMSE $18,205, MAE $13,087, R^2 = 0.940**. It explains 94% of the variance in sale prices, and a typical prediction is off by about 7.9%.
- **XGBoost and LightGBM** follow closely (R^2 0.932-0.933, MAE about $13.6-14.0k).
- **The Ridge baseline is competitive** (R^2 0.934). With well-engineered features and a log target, much of the

signal is close to linear. All the gaps are within about one CV standard deviation (about 0.01), so CatBoost's lead is real but small. Averaging the three boosters (R^2 0.938) is a robust alternative.

- **RMSE vs MAE:** RMSE ($18.2k) is about 40% above MAE ($13.1k), so a few houses have much larger errors than typical. Most of these are expensive homes (the top-quartile MAE is about $22k), which is consistent with the residual plots below.
- **Error by price band (CatBoost):** the MAE is $9.7k in the cheapest quartile and $22.5k in the most expensive. In percentage terms the errors are similar (6-8%) except for the cheapest homes (11%), which are the hardest to price in relative terms.

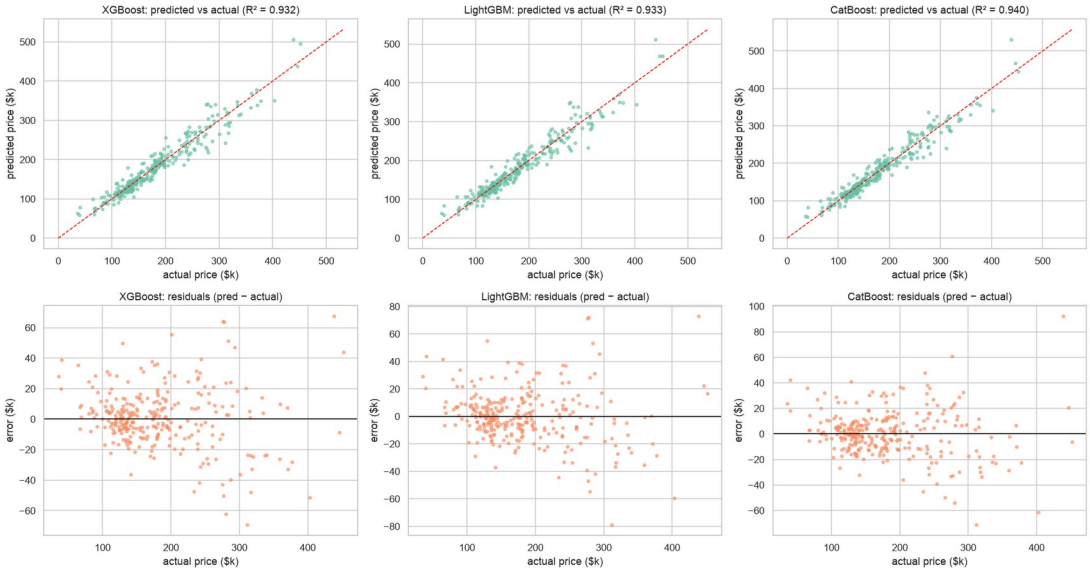


Figure 13 - Predicted vs actual (top) and residuals (bottom) on the test set

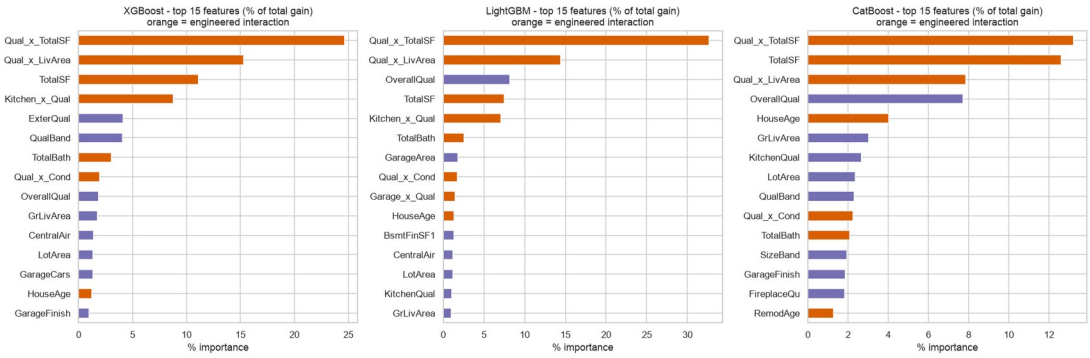


Figure 14 - Top-15 features per model (orange = engineered interactions)

What the models learned: all three rank **Qual_x_TotalSF**, **Qual_x_LivArea** and **TotalSF** at the top. These are quality and size combined, which confirms the EDA finding. They are followed by OverallQual, Kitchen_x_Qual, HouseAge, TotalBath and LotArea.

Conclusion: sale price is mainly driven by **how good, how big and how new** a house is, and by where it sits. CatBoost predicts it with R^2 = 0.94 and a typical error of about $13k (MAE).

Limitations: the data covers one city (Ames, 2006-2010) and 1,458 sales, so the model should be retrained before being used in another market. Possible next steps are stacking the boosters with Ridge and adding SHAP explanations for individual predictions.

Final note

Great job reaching the final stage of this mini project!

This is your chance to apply everything you've learned in a real-world setting. Data science isn't about getting it perfect — it's about exploring, experimenting, and growing through the process.

If you feel stuck, that's okay. Take a deep breath, break things down into smaller steps, and keep moving forward.

Explore boldly, think deeply — and trust yourself. You can do this — and you're closer than you think.

THE END

For learner use only